

# Implementasi Protokol AAA, Enkripsi Kolom, dan Tanda Tangan Digital pada Aplikasi TUMBUH

*Implementation of AAA Protocol, Column Encryption, and Digital Signature in the TUMBUH Application*

RAFIF MUHAMMAD FARRAS<sup>1\*</sup>, ADAM NAUFAL<sup>1</sup>, GHIFFARI BRAVIA HISHAM<sup>1</sup>

---

<sup>1</sup>Departemen Ilmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, IPB University, Bogor 16680

\*Penulis Korespondensi: Rafif Muhammad Farras; Tel: 081284314322; Surel: rafiffarras23@gmail.com

Adam Naufal; Tel: 081293625601; Surel: adaman507@gmail.com

Ghiffari Bravia Hisham; Tel: 081286714480; Surel: ghiffari.bravia.hisham@gmail.com

---

**Abstrak.** TUMBUH merupakan aplikasi web pelacak karier dan magang untuk komunitas akademik IPB University yang mengelola data sensitif mahasiswa, meliputi identitas, dokumen CV, surat lamaran, dan status seleksi. Meningkatnya ancaman siber dan berlakunya Undang-Undang Pelindungan Data Pribadi No. 27 Tahun 2022 mendorong penerapan kontrol keamanan terstruktur pada sistem semacam ini. Penelitian ini merancang dan mengimplementasikan kerangka keamanan berlapis pada TUMBUH berdasarkan protokol autentikasi, otorisasi, dan akuntansi yang diperkuat enkripsi simetris dan tanda tangan digital. Autentikasi menggunakan hashing kata sandi bcrypt, token berbasis JSON Web Token algoritma HS256, dan Google OAuth. Otorisasi menerapkan kendali akses berbasis peran dengan pemeriksaan kepemilikan data. Akuntansi dilakukan oleh mikroservis audit terpisah yang menjaga integritas catatan melalui rantai hash SHA-256. Enkripsi tingkat kolom Fernet melindungi kerahasiaan surat lamaran dari kebocoran basis data, sedangkan tanda tangan digital Ed25519 memberikan jaminan non-repudiasi pada peristiwa lamaran. Seluruh modul kriptografi terintegrasi langsung dalam alur utama aplikasi, bukan sebagai skrip terpisah, dan semua kunci dikelola melalui variabel lingkungan. Pengujian unit pada sembilan skenario, mencakup *roundtrip* enkripsi-dekripsi, deteksi pemalsuan tanda tangan, penolakan akses, dan penolakan *ciphertext* yang rusak, menghasilkan sembilan kelulusan. Hasil ini membuktikan kontrol yang diimplementasikan memenuhi sasaran kerahasiaan, integritas, dan akuntabilitas dalam cakupan yang diuji. Benchmark tambahan menunjukkan bahwa overhead terbesar berada pada jalur login karena verifikasi bcrypt, yaitu 276,4189 ms dibanding baseline sintesis 0,0002 ms, sedangkan overhead submit dan read tetap kecil, masing-masing 0,1224 ms dan 0,0625 ms. Enkripsi surat lamaran berukuran 1007 byte menghasilkan data tersimpan 1427 byte atau overhead 1,42x.

**Kata kunci.** autentikasi, enkripsi kolom, keamanan informasi, kendali akses berbasis peran, non-repudiasi, tanda tangan digital

**Abstract.** TUMBUH is a career and internship tracking web application for the IPB University academic community. It handles sensitive data, including student identities, CVs, application letters, and selection statuses. Indonesia's Personal Data Protection Law (Law No. 27/2022) makes structured security controls a requirement for systems that process such data. This paper documents the design and implementation of a multi-layer security framework for TUMBUH based on the AAA (Authentication, Authorization, Accounting) protocol, supported by symmetric encryption and digital signatures. Authentication uses bcrypt password hashing with automatic salting, JWT HS256 token management (60-minute access tokens, 7-day refresh cycles), and Google OAuth integration. Authorization uses a role-based access control (RBAC) model across three roles with ownership verification. A separate audit microservice handles

accounting and protects log integrity through SHA-256 hash chaining. Column-level Fernet encryption (AES-128-CBC + HMAC-SHA256) keeps application letters confidential, and Ed25519 digital signatures provide non-repudiation on application events. All cryptographic modules are integrated into the main application workflow rather than run as standalone scripts, and all keys are stored in environment variables with no hardcoded secrets. Unit testing covered nine scenarios: encryption-decryption roundtrips, signature forgery detection, access denial (HTTP 403), and corrupted ciphertext rejection. All nine passed, confirming that the controls meet the confidentiality, integrity, and accountability objectives within the tested scope. A supplementary benchmark showed that the largest overhead occurs in the login path because of bcrypt verification: 276.4189 ms compared with a synthetic baseline of 0.0002 ms. Submit and read overhead remained small at 0.1224 ms and 0.0625 ms, respectively. Encrypting a 1007-byte application letter produced a 1427-byte stored value, or a 1.42x storage overhead.

**Keywords.** authentication, column-level encryption, digital signature, information security, non-repudiation, role-based access control

---

## 1. PENDAHULUAN

Sistem informasi berbasis web yang mengelola proses rekrutmen dan magang semakin umum di lingkungan perguruan tinggi. TUMBUH adalah salah satunya, sebuah aplikasi yang menghubungkan mahasiswa IPB University dengan perusahaan rekanan dan admin platform dalam satu ekosistem daring. Mahasiswa dapat mencari lowongan, menyimpan daftar favorit, dan mengirimkan lamaran; HR perusahaan mengelola lowongan dan memantau proses seleksi; admin mengawasi keseluruhan sistem. Untuk menjalankan fungsi-fungsi tersebut, sistem menyimpan data yang cukup sensitif, yaitu identitas dan profil akademik mahasiswa, nomor induk mahasiswa, dokumen CV, isi surat lamaran, hingga riwayat status seleksi pelamar.

Karakteristik multi-peran sistem ini menghadirkan risiko keamanan yang spesifik. Kata sandi yang lemah dan tidak adanya pembatasan percobaan masuk membuka peluang tebak paksa (*brute force*) dan *credential stuffing*. Jika otorisasi tidak diverifikasi di setiap lapisan, data milik satu mahasiswa berpotensi diakses oleh mahasiswa lain atau oleh HR dari perusahaan yang tidak relevan. Kebocoran basis data pun tidak bisa diabaikan; surat lamaran yang tersimpan sebagai teks biasa akan langsung terbaca jika berkas basis data jatuh ke tangan yang salah. Yang lebih sulit ditangani adalah risiko penyangkalan tindakan: mahasiswa yang menyangkal pernah mengirimkan lamaran, atau perusahaan yang mengklaim tidak pernah mengubah status seleksi. Tanpa catatan audit yang tahan gangguan dan tanda tangan kriptografis pada setiap peristiwa, klaim semacam itu sangat sulit dibantah (Anderson 2020). Badan Siber dan Sandi Negara melaporkan peningkatan insiden kebocoran data di sektor pendidikan dalam beberapa tahun terakhir (Badan Siber dan Sandi Negara 2023), dan berlakunya Undang-Undang Pelindungan Data Pribadi No. 27 Tahun 2022 (Republik Indonesia 2022) menjadikan penerapan kontrol keamanan bukan lagi pilihan melainkan kewajiban hukum.

Keamanan informasi secara klasik diarahkan pada tiga sasaran utama, yaitu kerahasiaan, integritas, dan ketersediaan data, yang pada sistem multi-peran sering diperluas dengan akuntabilitas dan non-repudiasi (Anderson 2020; Stallings dan Brown 2017). Kerangka AAA, yang mencakup *Authentication*, *Authorization*, dan *Accounting*, menyediakan struktur operasional untuk mencapai sasaran-sasaran tersebut secara sistematis (Vollbrecht et al. 2000). Masing-masing komponen kerangka ini telah diteliti cukup mendalam. Model kendali akses berbasis peran dirumuskan oleh Sandhu et al. (1996) dan menjadi standar otorisasi pada sistem multi-pengguna. Penyimpanan kata sandi yang aman menggunakan bcrypt, sebuah fungsi hashing adaptif dengan

*salt* otomatis yang secara eksplisit dirancang tahan terhadap serangan tebak paksa (Provos dan Mazières 1999). Untuk enkripsi data, standar AES (National Institute of Standards and Technology 2001) menjadi fondasi, dan spesifikasi Fernet menggabungkan AES-128-CBC dengan HMAC-SHA256 untuk menghasilkan enkripsi terautentikasi (Katz dan Lindell 2014; Python Cryptographic Authority 2024). Tanda tangan digital Ed25519 berbasis Curve25519 dipilih atas dasar keunggulan kinerja, ukuran tanda tangan yang ringkas, dan sifat deterministiknya dibanding ECDSA (Bernstein et al. 2012; Josefsson dan Liusvaara 2017). Autentikasi berbasis token memanfaatkan JSON Web Token dengan algoritma HS256 (Jones et al. 2015) dan OAuth 2.0 untuk federasi identitas pihak ketiga (Hardt 2012). Panduan identitas digital dari NIST (2017) memberikan acuan tentang siklus hidup token yang aman, termasuk persyaratan masa berlaku dan mekanisme pencabutan.

Meskipun komponen-komponen tersebut sudah mapan secara individual, dokumentasi implementasi yang mengintegrasikan seluruhnya dalam satu alur kerja aplikasi web akademik masih jarang tersedia. Sebagian besar implementasi yang terdokumentasi hanya mencakup satu atau dua aspek, misalnya autentikasi berbasis token atau penerapan RBAC, tanpa menyertakan enkripsi tingkat kolom pada atribut data tertentu maupun tanda tangan digital per peristiwa bisnis. OWASP Top Ten (OWASP Foundation 2021) menempatkan *Broken Access Control* dan *Cryptographic Failures* sebagai dua kelemahan paling umum pada aplikasi web, keduanya bersesuaian langsung dengan ancaman yang dihadapi TUMBUH.

Penelitian ini bertujuan mengimplementasikan dan mendokumentasikan kerangka keamanan berlapis pada TUMBUH yang mencakup protokol AAA secara menyeluruh, enkripsi tingkat kolom berbasis Fernet, tanda tangan digital Ed25519 untuk non-repudiasi, dan rantai hash SHA-256 untuk integritas catatan audit. Seluruh komponen terintegrasi dalam alur kerja utama aplikasi dan diverifikasi melalui pengujian unit.

## 2. METODE

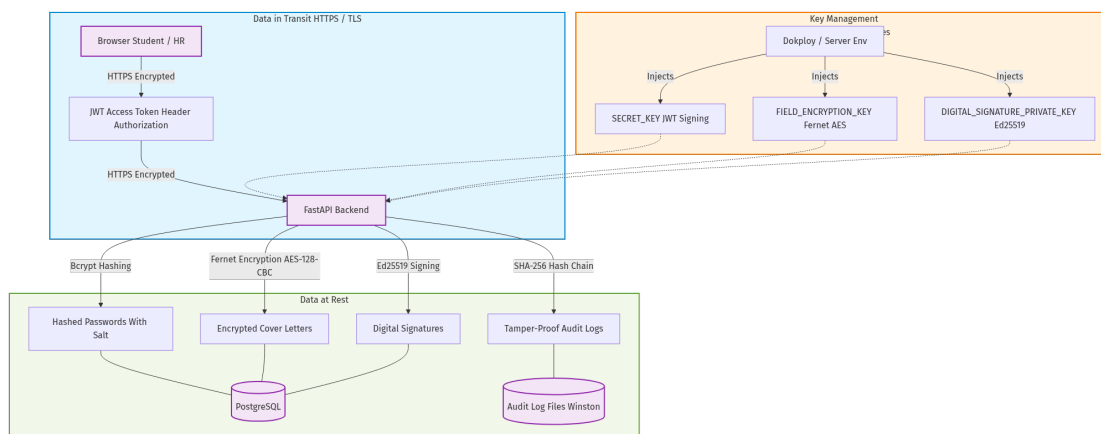
Penelitian ini menggunakan pendekatan studi kasus berbasis implementasi dalam konteks perkuliahan *Problem-Based Learning* (PBL) mata kuliah Keamanan Informasi (KOM1315) IPB University, Semester Genap 2025/2026.

### 2.1 Desain Penelitian

Pelaksanaan proyek mengikuti siklus berurutan: identifikasi aset dan pemodelan ancaman, perancangan kontrol keamanan, implementasi kode, integrasi ke alur kerja utama, dan pengujian unit. Tahap integrasi secara khusus mensyaratkan bahwa seluruh modul kriptografi menyatu dengan alur bisnis utama aplikasi, bukan dijalankan sebagai skrip terpisah yang dapat dilewati.

### 2.2 Deskripsi Sistem dan Arsitektur

TUMBUH menggunakan arsitektur tiga komponen. *Frontend* dibangun dengan React dan Vite, berkomunikasi dengan *backend* melalui antarmuka REST (Fielding 2000). *Backend* menggunakan FastAPI berbasis Python dengan ORM SQLAlchemy, yang menyediakan validasi masukan otomatis melalui skema Pydantic dan *dependency injection* terstruktur untuk penerapan kontrol otorisasi. Layanan audit berjalan sebagai mikroservis Node.js Express yang terpisah. PostgreSQL melayani kebutuhan basis data, dan komunikasi produksi dilindungi TLS 1.3 (Rescorla 2018). Gambar 1 menunjukkan aliran data antarkomponen beserta penempatan kontrol kriptografi.



**Gambar 1.** Arsitektur keamanan TUMBUH: aliran data antarkomponen, penempatan kontrol kriptografi, dan skema manajemen kunci.

### 2.3 Pemodelan Ancaman dan Pemetaan Kontrol

Pemodelan ancaman dilakukan menggunakan kerangka STRIDE untuk mengkategorikan ancaman secara sistematis. Enam kategori ancaman utama beserta kontrol yang ditetapkan dirangkum dalam Tabel 1.

**Tabel 1.** Pemetaan ancaman terhadap kontrol keamanan pada TUMBUH

| Ancaman                                    | Kontrol yang diterapkan                                                 |
|--------------------------------------------|-------------------------------------------------------------------------|
| Pencurian akun dan tebak paksa kata sandi  | <i>Password hashing</i> bcrypt, <i>rate limiting</i> , verifikasi surel |
| Akses tidak sah ke data pengguna lain      | RBAC, pemeriksaan kepemilikan, kontrol akses dokumen CV                 |
| Kebocoran data sensitif dari basis data    | Enkripsi tingkat kolom Fernet (AES-128-CBC + HMAC-SHA256)               |
| Manipulasi dan penyangkalan status lamaran | Tanda tangan digital Ed25519, jejak audit                               |
| Pemalsuan atau penghapusan catatan audit   | Rantai <i>hash</i> SHA-256 ( <i>tamper evidence</i> )                   |
| Kebocoran kunci kriptografi                | Variabel lingkungan, tanpa <i>hardcoded keys</i>                        |

### 2.4 Alat dan Bahan

Tabel 2 merangkum komponen dan pustaka yang digunakan. Pemilihan pustaka mengutamakan implementasi kriptografi yang sudah diaudit komunitas, bukan implementasi mandiri.

**Tabel 2.** Komponen dan pustaka utama beserta fungsi keamanannya

| Komponen          | Teknologi            | Fungsi keamanan                                    |
|-------------------|----------------------|----------------------------------------------------|
| <i>Frontend</i>   | React + Vite         | Penyimpanan token, permintaan terautentikasi       |
| <i>Backend</i>    | FastAPI + SQLAlchemy | Logika AAA, validasi masukan, kriptografi          |
| Audit             | Express + Winston    | Pencatatan audit dan rantai <i>hash</i>            |
| Basis data        | PostgreSQL           | Penyimpanan, <i>parameterized query</i>            |
| <i>Hashing</i>    | bcrypt 4.2.0         | <i>Hashing</i> kata sandi dengan <i>salt</i>       |
| Token             | python-jose 3.3.0    | Penerbitan dan verifikasi JWT (HS256)              |
| Kriptografi       | cryptography (PyCA)  | Enkripsi kolom (Fernet) dan tanda tangan (Ed25519) |
| OAuth             | google-auth 2.45.0   | Verifikasi token identitas Google                  |
| <i>Rate limit</i> | slowapi 0.1.9        | Pembatasan laju permintaan per IP                  |

## 2.5 Manajemen Kunci

Tidak ada kunci kriptografi yang ditanamkan langsung dalam kode sumber. Tiga rahasia utama dikelola melalui variabel lingkungan: SECRET\_KEY untuk penandatanganan JWT, FIELD\_ENCRYPTION\_KEY sebagai kunci simetris Fernet, dan DIGITAL\_SIGNATURE\_PRIVATE\_KEY sebagai *seed* derivasi kunci Ed25519. Berkas konfigurasi yang memuat rahasia dikecualikan dari repositori melalui .gitignore.

Pada lingkungan pengembangan, kunci Fernet dan *seed* Ed25519 dapat diturunkan secara deterministik dari SECRET\_KEY menggunakan SHA-256 untuk menyederhanakan konfigurasi awal, sedangkan pada produksi setiap kunci disuntikkan secara eksplisit dan independen (National Institute of Standards and Technology 2020). Pemisahan ini memastikan bocornya satu kunci tidak otomatis mengompromikan kunci lainnya.

## 2.6 Prosedur Pengujian

Pengujian dilakukan menggunakan pytest terhadap basis data SQLite *in-memory* sehingga tidak bergantung pada infrastruktur produksi. Sembilan skenario dirancang untuk memverifikasi tiga properti keamanan utama: kerahasiaan (tiga skenario), integritas tanda tangan (satu skenario), otorisasi (dua skenario), dan keamanan akun (tiga skenario). Keluaran pytest diarsipkan di folder 05\_Testing\_Logs pada repositori.

## 2.7 Prosedur Benchmark Kinerja

Untuk memenuhi kebutuhan analisis kuantitatif, benchmark tambahan dilakukan menggunakan skrip benchmark\_security.py. Benchmark ini sengaja tidak mengakses basis data dan tidak melakukan panggilan jaringan agar biaya yang terukur berfokus pada operasi keamanan inti: bcrypt, JWT, Fernet, dan Ed25519. Baseline *before* bersifat sintetis, yaitu jalur login dengan perbandingan kata sandi plaintext, jalur submit dengan surat lamaran plaintext, dan jalur read dengan pembacaan plaintext. Baseline *after* mengukur jalur aman: login menggunakan verifikasi bcrypt dan penerbitan JWT, submit menggunakan validasi JWT, enkripsi Fernet, dan penandatanganan Ed25519, sedangkan read menggunakan validasi JWT dan dekripsi Fernet.

Benchmark dijalankan dengan 1000 iterasi untuk operasi cepat dan 10 iterasi untuk operasi bcrypt karena bcrypt memang dirancang lebih mahal secara komputasi. Audit logging tidak dihitung dalam benchmark karena backend mengirim event audit secara asinkron; yang diukur adalah biaya kriptografi dan validasi token pada proses aplikasi utama.

Kode 1 memperlihatkan cuplikan inti skrip benchmark: pengukuran durasi tiap operasi, jalur aman untuk login, submit, dan read, serta perhitungan ukuran plaintext dan ciphertext. Ketiga jalur aman tersebut kemudian dibandingkan terhadap baseline plaintext.

```

1 def time_operation(name: str, iterations: int, fn):
2     durations = []
3     for _ in range(iterations):
4         start = time.perf_counter_ns()
5         fn()
6         elapsed_ms = (time.perf_counter_ns() - start) / 1_000_000
7         durations.append(elapsed_ms)
8     return {
9         "name": name,
10        "mean_ms": statistics.fmean(durations),
11        "median_ms": statistics.median(durations),
12        "p95_ms": percentile(durations, 0.95),
13    }
14
15 def login_crypto_path():
16     AuthService.verify_password(PASSWORD, password_hash)

```

```

17     AuthService.create_access_token(token_data)
18     AuthService.create_refresh_token(token_data)
19
20 def submit_crypto_path():
21     AuthService.decode_token(access_token)
22     security_service.encrypt_text(COVER_LETTER)
23     security_service.sign_payload(signature_payload)
24
25 def read_crypto_path():
26     AuthService.decode_token(access_token)
27     security_service.decrypt_text(encrypted_letter)
28
29 raw_bytes = len(COVER_LETTER.encode("utf-8"))
30 encrypted_bytes = len((encrypted_letter or "").encode("utf-8"))

```

**Kode 1.** Cuplikan benchmark untuk mengukur overhead login, submit, read, dan ukuran data terenkripsi

### 3. HASIL DAN PEMBAHASAN

#### 3.1 Autentikasi

Autentikasi pada TUMBUH berjalan melalui tiga jalur yang saling melengkapi. Jalur pertama berbasis surel dan kata sandi: saat pendaftaran, sistem memvalidasi format surel, memeriksa domain institusional (mahasiswa wajib menggunakan @ipb.ac.id), memeriksa duplikasi, melakukan *hashing* kata sandi, lalu menerbitkan token verifikasi surel sekali pakai yang disimpan dalam bentuk *hash* SHA-256 dengan masa berlaku 24 jam. Saat masuk, sistem memeriksa kecocokan kata sandi, status verifikasi surel, dan status keaktifan akun sebelum token JWT diterbitkan. Pemisahan kondisi ini disengaja agar akun yang belum diverifikasi atau sudah dinonaktifkan tidak dapat memperoleh token yang valid.

Token diterbitkan menggunakan algoritma HS256, dengan *access token* berlaku 60 menit dan *refresh token* berlaku tujuh hari. Sistem secara eksplisit memverifikasi bahwa *refresh token* tidak dapat digunakan sebagai *access token* untuk memanggil *endpoint* terlindungi. Untuk autentikasi melalui Google, token identitas yang diterima diverifikasi tanda tangannya, audiensnya, dan masa berlakunya sebelum identitas dipercaya (Hardt 2012; Lodderstedt et al. 2013). Tidak ada kata sandi Google yang disimpan di sisi TUMBUH.

#### 3.2 Otorisasi

Otorisasi diterapkan dalam dua lapisan. Lapisan pertama adalah pembatasan berbasis peran dengan tiga peran yaitu student, hr, dan admin, diimplementasikan sebagai pabrik dependensi `require_role()` pada Kode 2. Fungsi ini menolak permintaan dengan HTTP 403 jika peran tidak sesuai dan dikaitkan ke *endpoint* melalui *dependency injection* FastAPI, sehingga kontrol ini tidak bisa dilewati tanpa mengubah definisi *endpoint* itu sendiri.

```

1 def require_role(required_role: str):
2     def _check_role(
3         current_user: User = Depends(get_current_user)
4     ) -> User:
5         if current_user.role.value != required_role:
6             raise HTTPException(
7                 status_code=status.HTTP_403_FORBIDDEN,
8                 detail=f"Access denied. Required role: {required_role}",
9             )
10        return current_user
11    return _check_role

```

**Kode 2.** Pabrik dependensi RBAC yang menolak akses dengan HTTP 403 bila peran tidak sesuai

Lapisan kedua adalah pemeriksaan kepemilikan. HR hanya dapat mengelola pelamar dari perusahaannya sendiri, baik untuk pembaruan status individual maupun massal. Dokumen CV disimpan di direktori privat dan tidak pernah dilayani sebagai berkas statis, sehingga setiap akses melewati lapisan autentikasi dan otorisasi. Lapisan ini penting karena memiliki peran yang benar saja tidak cukup jika pengguna masih bisa mengakses data orang lain yang kebetulan berbagi peran yang sama (Sandhu et al. 1996).

### 3.3 Akuntansi dan Audit Logging

Pencatatan aktivitas dilakukan oleh mikroservis audit berbasis Express dan Winston yang menerima peristiwa dari *backend*. Pemanggilan bersifat *fire-and-forget* pada utas latar belakang sehingga kegagalan layanan audit tidak memblokir respons API utama. Peristiwa yang dicatat mencakup registrasi, masuk berhasil dan gagal, penyegaran token, pengiriman dan pembaruan lamaran, pelanggaran *rate limit*, serta kesalahan tak tertangani.

Integritas catatan dijaga dengan rantai *hash* SHA-256. Setiap peristiwa baru di-*hash* bersama *hash* peristiwa sebelumnya sebagaimana ditunjukkan pada Kode 3. Penghapusan atau modifikasi satu entri akan memutus rantai dan dapat dideteksi oleh fungsi verifikasi yang membaca *log* secara kronologis.

```

1 function chainAuditEvent(event) {
2   const previousHash = loadLastHash(); // 'GENESIS' jika pertama
3   const eventHash = crypto
4     .createHash('sha256')
5     .update(previousHash + ':' + canonicalJson(event))
6     .digest('hex');
7   return {
8     ...event,
9     previousHash,
10    eventHash,
11    integrityAlgorithm: 'SHA-256 hash chain',
12  };
13 }

```

**Kode 3.** Perantaraan hash SHA-256 pada tiap entri audit; modifikasi satu entri memutus rantai

### 3.4 Hashing Kata Sandi

Kata sandi tidak pernah disimpan dalam bentuk aslinya. *bcrypt* dipilih karena dua sifatnya yang saling melengkapi: *salt* otomatis per kata sandi yang mencegah serangan *rainbow table*, dan faktor biaya yang dapat disesuaikan (Provos dan Mazieres 1999). Pada faktor biaya 12 yang digunakan, setiap operasi *hashing* membutuhkan waktu sekitar 200–300 ms pada *hardware* umum, cukup menyulitkan serangan tebak paksa berskala besar tanpa terasa mengganggu pengguna. Faktor ini pun dapat dinaikkan di masa depan tanpa mengubah kode. Implementasinya ditunjukkan pada Kode 4. *Field* *hashed\_password* tidak dideklarasikan dalam skema respons API sehingga tidak mungkin bocor melalui *endpoint* manapun.

```

1 @staticmethod
2 def hash_password(password: str) -> str:
3     return _bcrypt.hashpw(
4         password.encode("utf-8"), _bcrypt.gensalt()
5     ).decode("utf-8")
6
7 @staticmethod

```

```

8 def verify_password(plain: str, hashed: str) -> bool:
9     return _bcrypt.checkpw(
10         plain.encode("utf-8"), hashed.encode("utf-8")
11     )

```

**Kode 4.** Operasi hashing dan verifikasi kata sandi menggunakan bcrypt dengan salt otomatis

### 3.5 Enkripsi Tingkat Kolom

Surat lamaran dienkripsi sebelum disimpan ke basis data menggunakan Fernet, yang menggabungkan AES-128-CBC dengan HMAC-SHA256 untuk memberikan enkripsi terautentikasi (Katz dan Lindell 2014; National Institute of Standards and Technology 2001). Modifikasi sekecil apapun pada *ciphertext* menyebabkan dekripsi gagal karena HMAC akan tidak cocok. Enkripsi tingkat kolom dipilih atas enkripsi penuh basis data karena granularitasnya; hanya kolom yang memerlukan perlindungan ekstra yang dienkripsi, sementara kolom lain tetap bisa dicari dan diindeks secara efisien.

Kode 5 menunjukkan fungsi enkripsi dan dekripsi. Nilai terenkripsi diberi awalan `enc:v1:` untuk menandai status enkripsi, mencegah enkripsi ganda, dan menyediakan jalur migrasi jika algoritma diganti. Di basis data, surat lamaran tersimpan sebagai *ciphertext*; di respons API, nilai dikembalikan setelah didekripsi. Kebocoran basis data tidak serta-merta mengungkap isi surat lamaran karena penyerang tetap memerlukan kunci enkripsi yang disimpan terpisah di luar basis data (Python Cryptographic Authority 2024).

```

1 ENCRYPTED_PREFIX = "enc:v1:"
2
3 def encrypt_text(self, value: str | None) -> str | None:
4     if value is None or value.startswith(ENCRYPTED_PREFIX):
5         return value
6     token = self._fernet.encrypt(
7         value.encode("utf-8")
8     ).decode("utf-8")
9     return f"{ENCRYPTED_PREFIX}{token}"
10
11 def decrypt_text(self, value: str | None) -> str | None:
12     if value is None or not value.startswith(ENCRYPTED_PREFIX):
13         return value
14     raw = value.removeprefix(ENCRYPTED_PREFIX).encode("utf-8")
15     try:
16         return self._fernet.decrypt(raw).decode("utf-8")
17     except InvalidToken:
18         return "[encrypted value could not be decrypted]"

```

**Kode 5.** Enkripsi tingkat kolom Fernet dengan awalan versi untuk migrasi algoritma yang aman

### 3.6 Tanda Tangan Digital

Setiap peristiwa kritis dalam siklus lamaran, yaitu pengiriman oleh mahasiswa dan perubahan status oleh HR, ditandatangani menggunakan Ed25519. Sebelum ditandatangani, *payload* dikanonikalisasi menjadi JSON dengan kunci terurut agar deterministik. *Payload*, tanda tangan, dan nama algoritma disimpan bersama catatan lamaran di basis data. Kode 6 menunjukkan fungsi penandatanganan dan verifikasi.

```

1 def sign_payload(self, payload: dict) -> str:
2     canonical = self._canonical_json(payload).encode("utf-8")
3     signature = self._private_key.sign(canonical)
4     return base64.urlsafe_b64encode(signature).decode("ascii")
5

```



```

6 def verify_signature(
7     self, payload: dict, signature: str
8 ) -> bool:
9     try:
10         raw = base64.urlsafe_b64decode(
11             signature.encode("ascii")
12         )
13         self._public_key.verify(
14             raw,
15             self._canonical_json(payload).encode("utf-8")
16         )
17         return True
18     except Exception:
19         return False

```

**Kode 6.** Penandatanganan payload kanonik dan verifikasi tanda tangan Ed25519 untuk jaminan non-repudiasi

*Endpoint* verifikasi yang hanya bisa diakses admin memungkinkan pemeriksaan keaslian tanda tangan kapan saja. Jika *payload* lamaran pernah dimodifikasi setelah penandatanganan, verifikasi akan gagal. Mahasiswa tidak dapat menyangkal pengiriman lamaran dan HR tidak dapat menyangkal perubahan status karena klaim tersebut dapat dibantah secara kriptografis (Bernstein et al. 2012; Josefsson dan Liusvaara 2017).

### 3.7 Kontrol Keamanan Pendukung

Sejumlah kontrol pelengkap diterapkan mengikuti prinsip *defense in depth* (Anderson 2020). *Security headers* X-Content-Type-Options, X-Frame-Options, dan Referrer-Policy ditambahkan pada setiap respons untuk memitigasi *MIME sniffing* dan *clickjacking*. *Rate limiting* berbasis IP membatasi 5 hingga 10 permintaan per menit pada *endpoint* autentikasi. CORS dikonfigurasi dengan daftar *origin* yang diizinkan secara eksplisit. Unggahan berkas divalidasi berdasarkan tipe MIME dan ukuran, dengan nama berkas yang dibangkitkan ulang. Verifikasi surel menggunakan token sekali pakai berbasis SHA-256 yang kedaluwarsa setelah 24 jam. Pencegahan injeksi SQL diperoleh melalui *parameterized query* pada ORM SQLAlchemy, dan validasi masukan dijamin oleh skema Pydantic di setiap *endpoint* (OWASP Foundation 2021).

### 3.8 Integrasi pada Alur Utama Aplikasi

Semua modul kriptografi tertanam langsung pada alur bisnis utama aplikasi. Kode 7 menunjukkan *endpoint* pengiriman lamaran, tempat lima lapis keamanan berjalan dalam satu transaksi yang berurutan.

```

1 @router.post("/applications")
2 async def submit_application(
3     data: ApplicationCreate,
4     background_tasks: BackgroundTasks,
5     user: User = Depends(require_role("student")),
6     db: Session = Depends(get_db),
7 ):
8     # 1. Enkripsi sebelum penyimpanan ke basis data
9     encrypted = security.encrypt_text(data.application_letter)
10    app = Application(
11        student_id=user.id, job_id=data.job_id,
12        application_letter=encrypted,
13    )
14    db.add(app); db.flush()
15
16    # 2. Tanda tangan digital untuk non-repudiasi

```

```

17     payload = {
18         "action": "SUBMIT", "student_id": user.id,
19         "job_id": data.job_id, "application_id": app.id,
20         "timestamp": datetime.utcnow().isoformat(),
21     }
22     app.signature = security.sign_payload(payload)
23     app.signature_payload = json.dumps(payload, sort_keys=True)
24     app.signature_algorithm = "Ed25519"
25     db.commit()
26
27     # 3. Audit logging di latar belakang (fire-and-forget)
28     background_tasks.add_task(
29         audit.log_event, "APPLICATION_SUBMITTED", payload
30     )
31
32     # 4. Respons dengan nilai terdekripsi
33     return ApplicationResponse(
34         id=app.id,
35         application_letter=security.decrypt_text(encrypted),
36     )

```

**Kode 7.** Alur pengiriman lamaran: validasi token dan peran, enkripsi kolom, tanda tangan digital, audit logging, dan dekripsi untuk respons

Urutan operasi ini bukan sekadar pilihan gaya. Enkripsi harus terjadi sebelum penyimpanan ke basis data; penandatanganan harus terjadi setelah `flush()` agar `application.id` sudah tersedia dalam *payload*; dan *audit logging* dijalankan di latar belakang agar kegagalannya tidak membatalkan transaksi utama.

### 3.9 Perbandingan Fitur Keamanan

Tabel 3 membandingkan fitur keamanan TUMBUH dengan dua profil implementasi yang umum dijumpai.

**Tabel 3.** Perbandingan fitur keamanan TUMBUH dengan profil implementasi yang umum

| Fitur keamanan                          | Sistem akademik konvensional | Aplikasi web umum | TUMBUH |
|-----------------------------------------|------------------------------|-------------------|--------|
| <i>Password hashing</i> adaptif         | Sebagian                     | ✓                 | ✓      |
| Token JWT <i>stateless</i>              | ×                            | ✓                 | ✓      |
| OAuth identitas pihak ketiga            | ×                            | Sebagian          | ✓      |
| RBAC dengan <i>ownership check</i>      | ×                            | Sebagian          | ✓      |
| Enkripsi tingkat kolom                  | ×                            | ×                 | ✓      |
| Tanda tangan digital per peristiwa      | ×                            | ×                 | ✓      |
| Audit <i>log</i> berantai hash          | ×                            | ×                 | ✓      |
| Manajemen kunci via variabel lingkungan | ×                            | Sebagian          | ✓      |

(✓ = diterapkan; × = tidak ada; Sebagian = diterapkan dengan variasi kualitas)

Enkripsi tingkat kolom dan tanda tangan digital per peristiwa adalah dua fitur yang paling membedakan pendekatan TUMBUH dari implementasi konvensional. Keduanya menjawab langsung dua ancaman yang paling sulit ditangani: kebocoran basis data dan penyangkalan tindakan.

### 3.10 Hasil Pengujian

Tabel 4 merangkum hasil pengujian unit. Seluruh sembilan skenario berhasil dalam satu kali eksekusi pytest.

**Tabel 4.** Ringkasan hasil pengujian unit terhadap sembilan skenario keamanan

| Skenario pengujian                                   | Hasil      |
|------------------------------------------------------|------------|
| <i>Roundtrip</i> enkripsi-dekripsi surat lamaran     | Lulus      |
| Deteksi pemalsuan tanda tangan digital               | Lulus      |
| Penonaktifan akun memblokir proses masuk berikutnya  | Lulus      |
| Penghapusan akun mensyaratkan kecocokan surel        | Lulus      |
| Penggantian kata sandi via <i>reset</i> surel        | Lulus      |
| Pengguna student ditolak pada sumber daya admin      | Lulus      |
| Pengguna admin diizinkan pada sumber daya admin      | Lulus      |
| Data terenkripsi tidak terbaca sebagai teks biasa    | Lulus      |
| <i>Ciphertext</i> yang dirusak ditolak saat dekripsi | Lulus      |
| <b>Total</b>                                         | <b>9/9</b> |

Skenario ketiga mengonfirmasi bahwa kondisi keaktifan akun diperiksa sebagai bagian dari autentikasi, tidak hanya saat akun dibuat. Skenario kedelapan dan kesembilan bersama-sama membuktikan properti enkripsi terautentikasi Fernet bekerja sesuai desain; data hanya terbaca dalam kondisi normal dan modifikasi sekecil apapun pada *ciphertext* menyebabkan penolakan. Skenario keenam dan ketujuh mengonfirmasi RBAC bekerja secara simetris untuk menolak dan mengizinkan akses secara tepat.

### 3.11 Analisis Kinerja Kontrol Keamanan

Tabel 5 menunjukkan hasil benchmark before-vs-after untuk tiga jalur yang paling relevan terhadap kontrol keamanan: login, submit lamaran, dan read lamaran. Angka *before* bukan berasal dari versi lama aplikasi, melainkan baseline sintetis untuk menunjukkan biaya tambahan dari operasi keamanan. Dengan demikian, tabel ini menjelaskan trade-off performa dari kontrol yang ditambahkan, bukan membandingkan dua rilis aplikasi yang berbeda.

**Tabel 5.** Benchmark before-vs-after operasi keamanan TUMBUH

| Use case    | Before mean (ms) | After mean (ms) | Delta (ms) |
|-------------|------------------|-----------------|------------|
| Login path  | 0,0002           | 276,4189        | 276,4187   |
| Submit path | 0,0002           | 0,1224          | 0,1222     |
| Read path   | 0,0001           | 0,0625          | 0,0624     |

Overhead terbesar berada pada jalur login. Pada benchmark komponen, verifikasi bcrypt membutuhkan rata-rata 273,085 ms. Biaya ini bukan efek samping yang tidak diinginkan, melainkan karakteristik desain bcrypt: proses hashing dan verifikasi dibuat mahal agar serangan tebak paksa menjadi lebih sulit dilakukan dalam skala besar. Karena login tidak terjadi sesering operasi baca data, overhead ratusan milidetik masih dapat diterima untuk cakupan proyek ini, terutama jika digabungkan dengan pembatasan laju permintaan pada endpoint autentikasi.

Sebaliknya, overhead jalur submit dan read relatif kecil. Submit path membutuhkan rata-rata 0,1224 ms setelah kontrol keamanan, yang mencakup validasi JWT, enkripsi Fernet terhadap surat lamaran, dan tanda tangan digital Ed25519. Read path membutuhkan rata-rata 0,0625

ms karena hanya melakukan validasi JWT dan dekripsi Fernet. Hasil ini menunjukkan bahwa bottleneck keamanan utama berada pada bcrypt, bukan pada enkripsi kolom atau tanda tangan digital.

**Tabel 6.** Rincian waktu operasi keamanan pada benchmark lokal

| Operasi                     | Mean (ms) | Median (ms) | P95 (ms) |
|-----------------------------|-----------|-------------|----------|
| bcrypt verify               | 273,085   | 272,612     | 277,061  |
| JWT encode access+refresh   | 0,037     | 0,033       | 0,055    |
| JWT decode access           | 0,034     | 0,033       | 0,045    |
| Fernet encrypt cover letter | 0,021     | 0,020       | 0,028    |
| Fernet decrypt cover letter | 0,019     | 0,018       | 0,024    |
| Ed25519 sign payload        | 0,041     | 0,040       | 0,046    |
| Ed25519 verify payload      | 0,123     | 0,122       | 0,128    |

### 3.12 Overhead Ukuran Data Terenkripsi

Selain waktu eksekusi, enkripsi tingkat kolom juga menambah ukuran data yang disimpan. Pada sampel surat lamaran berukuran 1007 byte, format terenkripsi dengan awalan `enc:v1:` menghasilkan nilai tersimpan sebesar 1427 byte. Hasil ini dirangkum pada Tabel 7.

**Tabel 7.** Overhead ukuran data setelah enkripsi surat lamaran

| Field        | Plaintext (byte) | Encrypted (byte) | Overhead         |
|--------------|------------------|------------------|------------------|
| Cover letter | 1007             | 1427             | 420 byte / 1,42x |

Kenaikan ukuran ini berasal dari metadata Fernet, seperti timestamp, initialization vector, HMAC, dan encoding base64. Untuk field teks pendek seperti surat lamaran, overhead 1,42x masih wajar. Namun jika enkripsi diperluas ke file besar seperti CV, strategi yang lebih tepat adalah enkripsi file atau envelope encryption dengan manajemen kunci yang lebih matang.

Pengujian ini terbatas pada *unit testing* dengan basis data *in-memory*. Beberapa vektor serangan seperti *timing attacks*, keamanan konfigurasi TLS di produksi, dan kerentanan di *frontend* belum tercakup dan memerlukan *integration testing* serta *penetration testing* untuk penilaian yang lebih menyeluruh.

## 4. SIMPULAN

Penelitian ini berhasil mengimplementasikan kerangka keamanan berlapis pada TUMBUH yang mengintegrasikan protokol AAA, enkripsi tingkat kolom Fernet, tanda tangan digital Ed25519, dan rantai *hash* SHA-256 dalam satu alur kerja yang kohesif. Autentikasi diterapkan melalui bcrypt, JWT HS256, dan Google OAuth; otorisasi melalui RBAC tiga peran yang dilengkapi pemeriksaan kepemilikan; dan akuntansi melalui mikroservis audit dengan rantai *hash* SHA-256. Kebocoran basis data tidak serta-merta mengungkap surat lamaran karena kolom tersebut dienkripsi, sedangkan penyangkalan peristiwa lamaran dapat dibantah dengan tanda tangan Ed25519 yang tersimpan bersama setiap catatan. Seluruh kunci dikelola melalui variabel lingkungan. Pengujian unit terhadap sembilan skenario menghasilkan sembilan kelulusan, membuktikan kontrol yang diimplementasikan memenuhi sasaran kerahasiaan, integritas, dan akuntabilitas dalam cakupan yang diuji. Analisis kinerja menunjukkan bahwa biaya terbesar berada pada login karena verifikasi bcrypt, yaitu 276,4189 ms, sedangkan overhead submit dan read tetap di bawah 1 ms, masing-masing 0,1224 ms dan 0,0625 ms. Enkripsi surat lamaran

menambah ukuran data dari 1007 byte menjadi 1427 byte atau 1,42x. Dengan demikian, kontrol keamanan utama memberikan peningkatan perlindungan yang signifikan dengan overhead yang terukur dan masih dapat diterima pada cakupan proyek ini. Untuk pengembangan selanjutnya, rotasi kunci berkala melalui mekanisme awalan versi, perluasan enkripsi ke atribut sensitif lain, autentikasi dua faktor, serta *integration testing*, *load testing*, dan *penetration testing* menjadi prioritas yang direkomendasikan.

## UCAPAN TERIMA KASIH

Penulis menyampaikan terima kasih kepada Dr. Shelve Nidya Neyman dan Endang P. Giri, S.Kom, M.Kom selaku dosen mata kuliah Keamanan Informasi (KOM1315) IPB University atas bimbingan selama pelaksanaan proyek ini.

## References

- Anderson R. 2020. *Security Engineering: A Guide to Building Dependable Distributed Systems*. Ed ke-3. Hoboken (US), Wiley.
- Badan Siber dan Sandi Negara. 2023. Laporan tahunan keamanan siber indonesia 2022. [internet]. [diacu 2026 Jun 15]. Tersedia dari: <https://www.bssn.go.id>.
- Bernstein D, Duif N, Lange T, Schwabe P, Yang B. 2012. High-speed high-security signatures. *J Cryptogr Eng*. 2(2):77–89. doi:10.1007/s13389-012-0027-1.
- Fielding R. 2000. *Architectural Styles and the Design of Network-based Software Architectures* [disertasi]. University of California, Irvine (US).
- Hardt D. 2012. The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force.
- Jones M, Bradley J, Sakimura N. 2015. Json web token (jwt). RFC 7519, Internet Engineering Task Force.
- Josefsson S, Liusvaara I. 2017. Edwards-curve digital signature algorithm (eddsa). RFC 8032, Internet Engineering Task Force.
- Katz J, Lindell Y. 2014. *Introduction to Modern Cryptography*. Ed ke-2. Boca Raton (US), CRC Press.
- Lodderstedt T, McGloin M, Hunt P. 2013. Oauth 2.0 threat model and security considerations. RFC 6819, Internet Engineering Task Force.
- National Institute of Standards and Technology. 2001. Advanced encryption standard (aes). Technical Report FIPS PUB 197, NIST.
- National Institute of Standards and Technology. 2017. Digital identity guidelines: Authentication and lifecycle management. Technical Report SP 800-63B, NIST.
- National Institute of Standards and Technology. 2020. Recommendation for key management, part 1: General. Technical Report SP 800-57 Part 1 Rev. 5, NIST.
- OWASP Foundation. 2021. Owasp top ten. [internet]. [diacu 2026 Jun 15]. Tersedia dari: <https://owasp.org/www-project-top-ten/>.
- Provos N, Mazieres D. 1999. A future-adaptable password scheme. Technical report, USENIX. Proc. USENIX Annual Technical Conference, FREENIX Track; Jun 6–11; Monterey (US).
- Python Cryptographic Authority. 2024. Fernet (symmetric encryption). [internet]. [diacu 2026 Jun 20]. Tersedia dari: <https://cryptography.io/en/latest/fernet/>.
- Republik Indonesia. 2022. Undang-undang nomor 27 tahun 2022 tentang perlindungan data pribadi. Lembaran Negara RI Tahun 2022 Nomor 196.
- Rescorla E. 2018. The transport layer security (tls) protocol version 1.3. RFC 8446, Internet

Engineering Task Force.

Sandhu R, Coyne E, Feinstein H, Youman C. 1996. Role-based access control models. *Computer*. 29(2):38–47. doi:10.1109/2.485845.

Stallings W, Brown L. 2017. *Computer Security: Principles and Practice*. Ed ke-4. Boston (US), Pearson.

Vollbrecht J, Calhoun P, Farrell S, Gommans L, Gross G, de Bruijn B, de Laat C, Holdrege M, Spence D. 2000. Aaa authorization framework. RFC 2904, Internet Engineering Task Force.